

Algorithm 1 Search Procedure on an SSD-based Graph

```

1: procedure SEARCH( $G$ )
2:    $q \leftarrow$  query vector,  $p \leftarrow$  entry node
3:    $L \leftarrow$  search length,  $k \leftarrow$  top- $k$  vectors to search
4:    $S \leftarrow \{p\}$  ▷ candidate set
5:    $E \leftarrow \emptyset$  ▷ result set
6:    $u \leftarrow p$  ▷ exploring node
7:   Initialize PQ lookup table for  $q$ .
8:   while  $u \neq \text{null}$  do
9:      $(x, N) \leftarrow$  Read (vector, neighbor) of  $u$  ▷ On SSD
10:    set  $u$  as explored in  $S$ 
11:     $E.\text{insert}(u, \text{dist}(x, q))$ 
12:    for  $v \in N$  do
13:       $S.\text{insert}(v, \text{PQ-dist}(v, q))$ 
14:       $S.\text{resize}(L)$  ▷ Remove farthest candidates
15:    end for
16:     $u \leftarrow$  the nearest unexplored node in  $S$ 
17:  end while
18:  return top- $k$  nearest neighbor in  $E$ .
19: end procedure

```

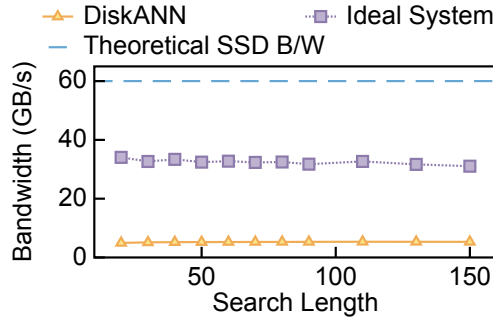


Fig. 2. Limited bandwidth utilization of DiskANN. We measure the SSD read bandwidth of DiskANN. We also compute its ideal bandwidth, assuming that all disk read latency can be hidden with computation (Ideal System). The raw bandwidth of our SSDs is shown in the blue dashed line.

search process terminates, and the top- k nearest nodes in E would be returned as the approximate nearest neighbors.

To reduce the number of fetching full-precision vectors, many on-disk graph-based ANNS systems keep a copy of in-memory compressed vectors using product quantization (PQ) [30, 34, 49], and use PQ distance as an approximation when inserting neighbors to the candidate set (Line 13).

3 Motivation

3.1 CPU Limits ANNS Cost-Effectiveness

Current disk-based graph ANNS systems fail to fully exploit the performance of SSDs. DiskANN [30] is a typical graph-based ANNS on SSDs. We measure its SSD read bandwidth during the search (§5.1

details the configuration of the experiments; for a more accurate estimation of the computing power, hyper-threading is disabled in this experiment to prevent interference between hyper-threads). Figure 2 shows that DiskANN achieves only 6 GB/s of SSD bandwidth, only 60% of the theoretical maximum bandwidth (10 GB/s) of the SSD.

Although DiskANN spends time on I/O waiting, we find that overlapping all of the I/Os still shows undesirable performance. In our experiment, a DiskANN worker thread spends approximately 1/6 of its time computing and 5/6 reading from the SSD. This is because DiskANN traverses the graph serially. Even with perfect I/O-computation overlap, where no CPU time is wasted waiting for I/O, the increased computation still cannot fully utilize the SSD bandwidth (*Ideal System* in Figure 2). This is because, as the *Ideal System* spends 6× of the CPU time on computing compared to DiskANN, it can only achieve 6× of the SSD bandwidth, which equals ~36 GB/s. The bandwidth of the ideal system still cannot reach the bandwidth upper bound of 6 SSDs (our configuration, 60 GB/s) because of the limited computing power of the CPU.

Our analysis shows that **CPU limits the high-throughput performance of ANNS**. This seems contrary to previous thoughts that the SSD is the dominating factor in disk-based ANNS [19, 30, 49]. This difference arises because previous works mainly focus on latency-critical scenarios. In these scenarios, assigning one CPU core per SSD is sufficient, because the latency of synchronous SSD reads is longer than the computation time, thus limiting the overall latency. However, to achieve high throughput, the number of CPU cores must match the SSD I/O bandwidth. For example, storage servers may have 6 – 24 SSDs [9, 14]. As a result, fully saturating the bandwidth requires approximately 94 – 375 CPU cores (Our 56-core server can serve the bandwidth of ~3.6 SSDs), which is difficult to achieve in a single server.

The insufficient CPU computing power limits the cost-effectiveness of ANNS. As CPUs on one server cannot saturate the bandwidth of the SSDs, more servers are needed to reach the same QPS, thus increasing the cost of deploying the ANNS service (§5.5).

3.2 Opportunity: GPU for ANNS

Based on the observation above, we use GPU instead of CPU for high-throughput and cost-effective ANNS, because of the strong computing power of the GPU. Due to its highly parallelized hardware design, GPU outperforms CPU in both FLOPS and memory bandwidth. For example, NVIDIA A100 can achieve 19.5 TFLOPS for 32-bit floating numbers, and 1935 GB/s memory bandwidth [6]. In contrast, CPUs have much lower FLOPS (1.25 TFLOPS for the 28-core CPU used in our evaluation [2]) and DRAM read bandwidth (typically ~30 GB/s for each DRAM).

Existing systems using GPU for ANNS show limitations for exploiting multiple SSDs. They can be categorized into two types:

(1) *CPU-centric, GPU-assisted architecture* [47, 56]. Only vector computation (e.g., distance) is offloaded to GPU; the main search process is still executed on CPU. Although this is feasible for cluster-based [56] indexes due to their high compute density of intra-cluster vectors, this architecture suffers from the performance bottleneck in graph-based indexes, which is not that compute-intensive compared to cluster indexes.

(2) *GPU-only architecture* [41, 48, 57]. All the search tasks are executed on GPU, including graph traversal and vector computation. It eliminates the bottleneck of CPU search. However, it suffers from limited GPU memory and the inbound transfer bandwidth of GPU.

To handle these limitations, we propose a *GPU-centric, CPU-assisted architecture*, where the search task is executed on GPU, but CPU is utilized to assist data transfer, which is possible to eliminate the bottleneck of limited in-bound transfer bandwidth. However, this architecture still faces several challenges:

3.3 Challenges

CHALLENGE 1. *The memory-consuming search process limits the concurrency of the system, which is required by both GPU and SSD.*

Both GPU and SSD are highly concurrent in their designs. GPUs achieve high computing power with a large number of computing threads. For example, the NVIDIA A100 GPU utilizes 108 streaming multiprocessors (SMs), and each SM supports 2048 threads running concurrently [6]. In addition, modern SSDs achieve high bandwidth by processing multiple requests simultaneously. As a result, combining GPU and SSD requires a highly concurrent system design.

However, existing graph-based ANNS systems cannot support high concurrency because of their high memory consumption. GPU memory is limited (e.g., 40 GB memory in our setup), and most of its memory ($\sim 75\%$) is used to store the compressed dataset (PQ codes). Therefore, the memory left for each query is scarce. However, most current implementations for graph-based ANNS are CPU-oriented [21, 30, 40] and use memory-consuming per-query data structures. Current GPU-based ANNS systems inherit this design [48, 57]. Consequently, these structures consume a significant amount of memory (28 – 45% of the memory usage for different implementations, see §4.2 and §5.3). As a result, the limited GPU memory restricts the maximum number of concurrent queries (a maximum of 50% reduction), further preventing the full utilization of the GPU and SSD.

CHALLENGE 2. *To mitigate the GPU-side bandwidth bottleneck, how to effectively use CPU to assist data transfer is under-exploited.*

GPU-only methods use *GPU direct-access* such as BaM [42], GPUDirect Storage (GDS) [7], to direct transfer from SSDs to the GPU. However, applying GPU direct-access cannot achieve desirable performance for graph-based ANNS. As Figure 3(a) indicates, a baseline system with GPU direct-access can only achieve $\sim 40\%$ of GUSTANN's maximum performance.

By analysis, we find that the CPU-centric PCIe topology limits efficient GPU access to SSDs. All PCIe devices are first connected to the root complex, which connects to the CPU and DRAM through high-bandwidth links. Typically, devices lack direct connections. Such a layout results in asymmetric transfer bandwidth between SSDs and the GPU. As the number of SSDs grows, their aggregated bandwidth may exceed the inbound bandwidth to the GPU, which becomes a bottleneck. Figure 6(a) gives a detailed example.

GPU direct-access suffers from this asymmetric bandwidth. Take BaM [42] as an example. As shown in Figure 3(b), when adopting BaM as the I/O engine of the ANNS with 6 SSDs, the effective bandwidth of SSD only reaches 27% of its theoretical maximum. This is because GPU direct-access methods need to read full SSD pages through the PCIe link, whose bandwidth is only $\sim 40\%$ of the maximum bandwidth as indicated in the figure.

CHALLENGE 3. *Load imbalance exists among SSDs with a RAID-0 layout, which degrades the overall search performance.*

To maximize the bandwidth of multiple SSDs, a natural approach is to distribute SSD pages in a RAID 0 layout. Although this layout distributes reads evenly (and so is the bandwidth) in our datasets, it still results in load imbalance in terms of SSD access latency during the search. We measure the SSD read latency with the RAID 0 layout in the search. As shown in Figure 3(c), the maximum 99% latency is $2.17\times$ higher than the minimum. Such high tail latency divergence is unacceptable for search on the GPU due to the bucket effect: the GPU prefers batching for high concurrency, but the long tail latency of a single query slows down the entire batch. As a result, both the SSD bandwidth utilization and the search performance drop because of the load imbalance (See §5.3.3).

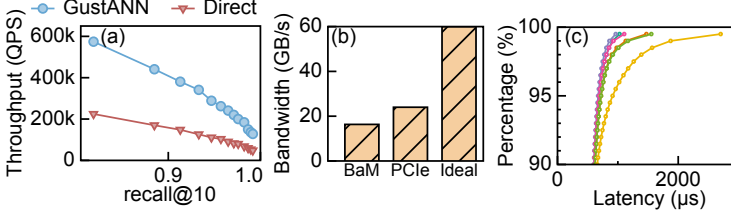


Fig. 3. Design challenges. (a) Throughput comparison of GustANN and the *GPU direct-access* solution (*Direct* in the figure). (b) The SSD bandwidth comparison of BaM (a recent GPU direct-access solution), and the theoretical maximum (*Ideal* in the figure). *PCIe* is the maximum PCIe transfer bandwidth of the GPU. (c) The distribution of the SSDs' read latency. Each line indicates an SSD.

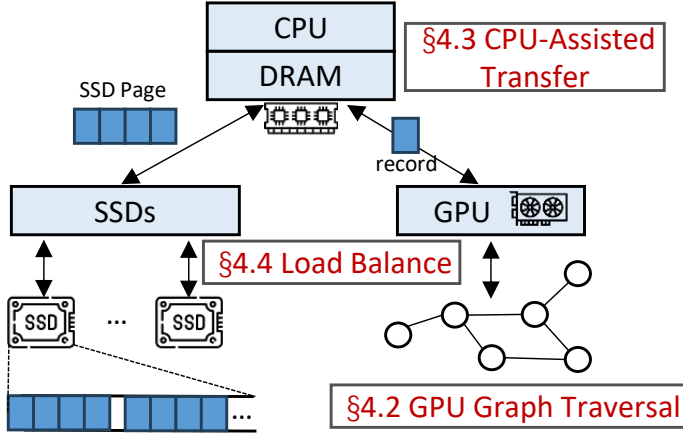


Fig. 4. GustANN architecture.

4 Design

4.1 Overview

GUSTANN uses a *GPU-centric, CPU-assisted architecture* to achieve high-throughput and cost-effective ANNS. Its overall architecture is shown in Figure 4.

4.1.1 Data Placement. The data placement on disks of GUSTANN is similar to DiskANN [30] (§2.2), which stores vectors and neighbors on disks. In addition, GUSTANN uses multiple SSDs, and the data is stored in a RAID 0 scheme. To tackle the load balance issue (Challenge 3), we leverage a pivot search (§4.4) and store a pivot graph in GPU memory. Also, most data structures (e.g., compressed vectors) are moved from DRAM to GPU memory.

The main usage of DRAM in GUSTANN is the bouncing buffers in the CPU-assisted transfer to enable selective transfer from SSD to GPU. Instead of GPU direct-access, this method can mitigate the bottleneck of the PCIe bandwidth for the GPU (§4.3). GUSTANN mainly uses DRAM for its bandwidth, and its capacity demand is much smaller compared to other DRAM-based billion-scale ANNS systems, thus reducing costs for servers (§5.5).